



Personal

Open source

Business

Explore

Pricing

Blog

Support

This repository

Search

Sign in

Sign up

 hangtwenty / **dive-into-machine-learning**

Watch

319

Star

5,086

Fork

587

<> Code

Issues 4

Pull requests 0

Pulse

Graphs

Dive into Machine Learning with Python Jupyter notebook and scikit-learn <http://hangtwenty.github.io/dive-into-machine-learning>

119 commits

2 branches

0 releases

16 contributors

Branch: master ▾

New pull request

New file

Find file

HTTPS ▾

<https://github.com/hangtw>

Download ZIP

hangtwenty Fix a minor grammatical error ...

Latest commit e70bade 26 days ago

.travis.yml	[travis] white list thetalkingmachines	3 months ago
LICENSE	License: CC Attribution 4.0 International	3 months ago
README.md	Fix a minor grammatical error	26 days ago

README.md

Dive into Machine Learning



Hi there! This guide is for you:

- You're new to Machine Learning.

- **You know Python.** (At least the basics! If you want to learn Python, try [Dive Into Python](#).)

I learned Python by hacking first, and getting serious *later*. I wanted to do this with Machine Learning. If this is your style, join me in getting a bit ahead of yourself.

Note: There are several fields within "Data" and Machine Learning is just one. It's good to know the context: [What is the difference between Data Analytics, Data Analysis, Data Mining, Data Science, Machine Learning, and Big Data?](#)

Get your feet wet!

I suggest you get your feet wet ASAP. You'll boost your confidence.

Tools you'll need

- [Python](#). Python 3 is the best option.
- [IPython and the Jupyter Notebook](#). (FKA IPython and IPython Notebook.)
- Some scientific computing packages:
 - numpy
 - pandas
 - scikit-learn
 - matplotlib

You can install Python 3 and all of these packages in a few clicks with the [Anaconda Python distribution](#). Anaconda is popular in Data Science and Machine Learning communities.

If you're using Python 2.7, don't worry. You don't have to migrate to Python 3 just for this guide. Also, if you're using pip/virtualenv instead of Anaconda, that's alright too! Cf. [conda vs. pip vs. virtualenv](#) if you're curious.

Let's go!

Learn how to use IPython Notebook (5-10 minutes). (You can learn by screencast instead.)

Now, follow along with this brief exercise (10 minutes): **An introduction to machine learning with scikit-learn**. Do it in `ipython` or IPython Notebook. It'll really boost your confidence.



"I'll wait."

What just happened?

You just classified some hand-written digits using [scikit-learn](#). Neat huh?

[scikit-learn](#) is the go-to library for machine learning in Python. Some recognizable logos use it, including Spotify and [Evernote](#). Machine learning is hard. You'll be glad your tools are easy to work with.

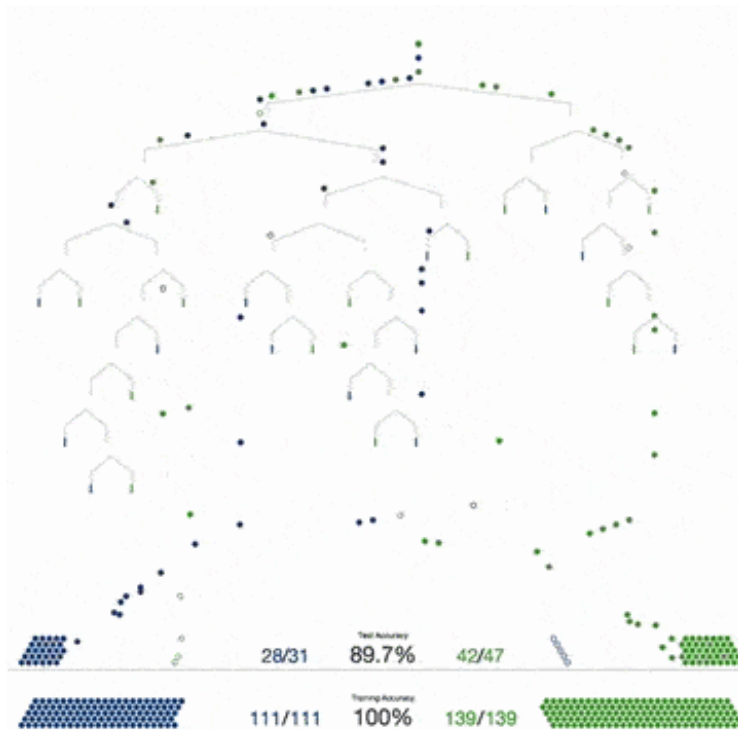
I encourage you to look at the [scikit-learn](#) homepage and spend about 5 minutes looking over the names of the strategies

(Classification, Regression, etc.), and their applications. Don't click through yet! Just get a glimpse of the vocabulary.

Dive in

A Visual Introduction to Machine Learning

Let's learn a bit more about Machine Learning, and a couple of common ideas and concerns. Read "[A Visual Introduction to Machine Learning, Part 1](#)" by Stephanie Yee and Tony Chu.



It won't take long. It's a beautiful introduction ... Try not to drool too much!

A Few Useful Things to Know about Machine Learning

OK. Let's dive deeper.

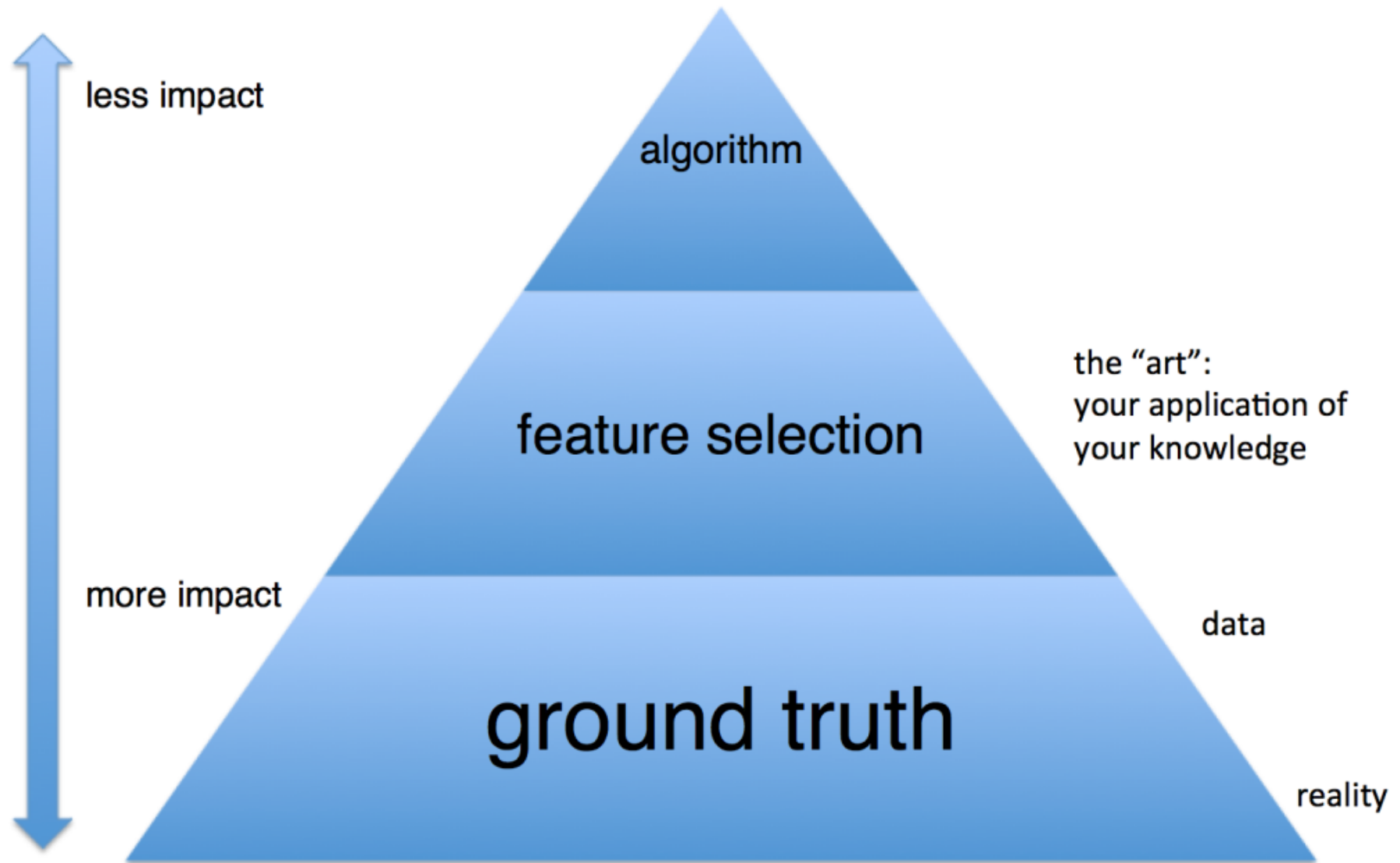
Read "[A Few Useful Things to Know about Machine Learning](#)" by Prof. Pedro Domingos. It's densely packed with valuable information, but not opaque. The author understands that there's a lot of "black art" and folk wisdom, and they invite you in.

Take your time with this one. Take notes. Don't worry if you don't understand it all yet.

The whole paper is packed with value, but I want to call out two points:

- **Data alone is not enough.** This is where science meets art in machine-learning. Quoting Domingos: "... the need for knowledge in learning should not be surprising. Machine learning is not magic; it can't get something from nothing. What it does is get more from less. Programming, like all engineering, is a lot of work: we have to build everything from scratch. Learning is more like farming, which lets nature do most of the work. Farmers combine seeds with nutrients to grow crops. Learners combine knowledge with data to grow programs."
- **More data beats a cleverer algorithm.** Listen up, programmers. We like cool tools. Resist the temptation to reinvent the wheel, or to over-engineer solutions. Your starting point is to [Do the Simplest Thing that Could Possibly Work](#). Quoting Domingos: "Suppose you've constructed the best set of features you can, but the classifiers you're getting are still not accurate enough. What can you do now? There are two main choices: design a better learning algorithm, or gather more data. [...] As a rule of thumb, a dumb algorithm with lots and lots of data beats a clever one with modest amounts of it. (After all, machine learning is all about letting data do the heavy lifting.)"

So **knowledge** and **data** are critical. Focus your efforts on those, before fussing about algorithms. In practice, this means that unless you *have* to increase complexity, you should continue to [Do Simple Things](#); don't rush to neural networks just because they're cool. To improve your model, get more data and use your knowledge of the problem to manipulate the data. You should spend most of your time on these steps. Only optimize your choice of algorithms after you've got enough data, and you've processed it well.



(Chart inspired by a slide from Alex Pinto's talk, "Secure Because Math: A Deep-Dive on ML-Based Monitoring".)

Talking Machines

Subscribe to **Talking Machines**, a podcast about machine learning. It's great. It's a low-effort, high-yield way to learn more.

I suggest this listening order:

- Start with the "[Starting Simple](#)" episode. It supports what we read from Domingos. Ryan Adams talks about starting simple, as discussed above. Adams also stresses the importance of feature engineering. Feature engineering is an exercise of the "knowledge" Domingos writes about.
- Then, start over from the first episode

Play to learn

Pick **one or two** of these IPython Notebooks and play along.

- [Face Recognition on a subset of the Labeled Faces in the Wild](#)
- [Machine Learning from Disaster](#): Using Titanic data, "Demonstrates basic data munging, analysis, and visualization techniques. Shows examples of supervised machine learning techniques."
- [Election Forecasting](#): A replication of the model Nate Silver used to make [predictions about the 2012 US Presidential Election for the New York Times](#))
- [An example Machine Learning notebook](#): "let's pretend we're working for a startup that just got funded to create a smartphone app that automatically identifies species of flowers from pictures taken on the smartphone. We've been tasked by our head of data science to create a demo machine learning model that takes four measurements from the flowers (sepal length, sepal width, petal length, and petal width) and identifies the species based on those measurements alone."
- ClickSecurity's "data hacking" series (thanks [hummus!](#))
 - [Detect Algorithmically Generated Domains](#)
 - [Detect SQL Injection](#)
 - [Java Class File Analysis](#): is this Java code malicious or benign?
- If you want more of a data science bent, pick a notebook from [this excellent list of Data Science ipython notebooks](#).

"Continually updated Data Science Python Notebooks: Spark, Hadoop MapReduce, HDFS, AWS, Kaggle, scikit-learn, matplotlib, pandas, NumPy, SciPy, and various command lines."

- Or more generic tutorials/overviews ...
 - [Tutorial introduction to machine learning with sklearn](#)
 - [An Introduction to Supervised Learning via Scikit Learn](#)
 - [An Introduction to Unsupervised Learning via Scikit Learn](#)

There are more places to find great IPython Notebooks:

- [A Gallery of Interesting IPython notebooks \(wiki page on GitHub\): Statistics, Machine Learning and Data Science](#)
- [Fabian Pedregosa's larger, automatic gallery](#)

Immerse yourself

Pick one of the courses below and do the whole thing.

Recommended course

[Prof. Andrew Ng's *Machine Learning*](#) is a free online course I've seen recommended often. And emphatically.

It's helpful if you decide on a pet project to play around with, as you go, so you have a way to apply your knowledge. You could use one of these [Awesome Public Datasets](#). And remember, IPython Notebook is your friend.

Also, you should grab an in-depth textbook to use as a reference. The two best options are [Understanding Machine Learning](#) and [Elements of Statistical Learning](#). You'll see these recommended as reference textbooks. I favor *UML*, but here's [context and comparison](#). Download both books, they're free.

Busy schedule? Read [Ray Li's review of this course](#) for some helpful tips.

Other courses

Here are some other free online courses I've seen recommended. (Machine Learning, Data Science, and related topics.)

- [Machine Learning](#) by Prof. Pedro Domingos of the University of Washington. Domingos wrote the paper "[A Few Useful Things to Know About Machine Learning](#)", from earlier in this guide. (Thanks to [paperwork on Hacker News](#).)
- Data science courses as IPython Notebooks:
 - [Practical Data Science](#)
 - [Learn Data Science](#) (an entire self-directed course!)
 - Supplementary material: [donnemartin/data-science-ipython-notebooks](#). "Continually updated Data Science Python Notebooks: Spark, Hadoop MapReduce, HDFS, AWS, Kaggle, scikit-learn, matplotlib, pandas, NumPy, SciPy, and various command lines."
- Kevin Markham's video series, [Intro to Machine Learning with scikit-learn](#), starts with what we've already covered, then continues on at a comfortable place. After the videos you could do Markham's [General Assembly's Data Science course](#). Interactive. (Older versions: [7](#), [5](#), [4](#), [3](#).) Markham's course is also offered in-person in Washington, DC.
- Prof. Mark A. Girolami's [Machine Learning Module \(GitHub Mirror\)](#). Good for people with a strong mathematics background.
- Surveys of Data Science courseware (a bit more Choose Your Own Adventure)
 - Check out [Jack Golding's survey of Data Science courseware](#). Includes Coursera's [Data Science Specialization](#) with 9 courses in it. The Specialization certificate isn't free, but you can take the courses 1-by-1 for free if you don't care about the certificate. The survey also covers [Harvard CS109](#) which I've seen recommended elsewhere.
 - [Another epic Quora thread: How can I become a data scientist?](#)
 - Data Science Weekly's [Big List of Data Science Resources](#) has a [List of Data Science MOOCs](#)
- [Data Science \(Harvard CS109\)](#)
- [Advanced Statistical Computing \(Vanderbilt BIOS8366\)](#).. Interactive (lots of IPython Notebook material)

Learn Pandas well

If you're focusing on Python, you should get more familiar with Pandas.

- **Essential:** [10 Minutes to Pandas](#)
- **Essential:** [Things in Pandas I Wish I'd Had Known Earlier \(IPython Notebook\)](#)
- [Useful Pandas Snippets](#)
- Here are some docs I found especially helpful as I continued learning:
 - [Cookbook](#)
 - [Data Structures](#), esp. [DataFrame](#) section
 - [Reshaping by pivoting DataFrames](#)
 - [Computational tools and StackExchange thread: "What is covariance in plain language?"](#)
 - [Group By \(split, apply, and combine DataFrames\)](#)
 - [Visualizing your DataFrames](#)

Cheat sheets

Bookmark these cheat sheets:

- [scikit-learn algorithm cheat sheet](#)
- [Metacademy: a package manager for \[machine learning\] knowledge](#). A mind map of machine learning concepts, with great detail on each.
- [Matplotlib / Pandas / Python cheat sheets](#).

More topics

More Data Science materials

Not repeating the materials mentioned above, here are some more Data Science resources:

- **Extremely accessible data science book:** *Data Smart* by John Foreman
- **An entire self-directed course in Data Science, as a IPython Notebook**
- [Data Science Workflow: Overview and Challenges](#) (read the article *and also* the comment by Joseph McCarthy)
- Fun little IPython Notebook: [Web Scraping Indeed.com for Key Data Science Job Skills](#)
- Swami Chandrasekaran's "[Becoming a Data Scientist](#)" is a concise, printable picture of a data science curriculum

Bayesian Statistics and Machine Learning

From the "[Bayesian Machine Learning](#)" overview on Metacademy:

... Bayesian ideas have had a big impact in machine learning in the past 20 years or so because of the flexibility they provide in building structured models of real world phenomena. Algorithmic advances and increasing computational resources have made it possible to fit rich, highly structured models which were previously considered intractable.

You can learn more by studying one of the following resources. Both resources use Python, [PyMC](#), and Jupyter Notebooks.

- The **free book**, *Probabilistic Programming and Bayesian Methods for Hackers*. Made with a "computation/understanding-first, mathematics-second point of view." It's available in print too!
- [Bayesian Modelling in Python](#)

Questions, answers, chats

For now, the best StackExchange site is stats.stackexchange.com – *machine-learning*. (There's also datascience.stackexchange.com, but it's still in Beta.) And there's [/r/machinelearning](https://r/machinelearning). There are also many relevant discussions on Quora, for example: [What is the difference between Data Analytics, Data Analysis, Data Mining, Data Science, Machine Learning, and Big Data?](#)

You should also join the Gitter channel for scikit-learn!

For help and community in meatspace, seek out meetups. Data Science Weekly's [Big List of Data Science Resources](#) may help you.

Assorted Opinions and Other Resources

The rest of the stuff that might not be structured enough for a course, but seems important to know.

Risks

"Machine learning systems automatically learn programs from data." Pedro Domingos, in ["A Few Useful Things to Know about Machine Learning."](#) The programs you generate will require maintenance. Like any way of creating programs faster, you can rack up technical debt.

Here is the abstract of [Machine Learning: The High-Interest Credit Card of Technical Debt](#):

Machine learning offers a fantastically powerful toolkit for building complex systems quickly. This paper argues that it is dangerous to think of these quick wins as coming for free. Using the framework of technical debt, we note that it is remarkably easy to incur massive ongoing maintenance costs at the system level when applying machine learning. The goal of this paper is highlight several machine learning specific risk factors and design patterns to be avoided or refactored where possible. These include boundary erosion, entanglement, hidden feedback loops, undeclared consumers, data dependencies, changes in the external world, and a variety of system-level anti-patterns.

If you're following this guide, you should read that paper. You can also [listen to a podcast episode](#) interviewing one of the authors of this paper.

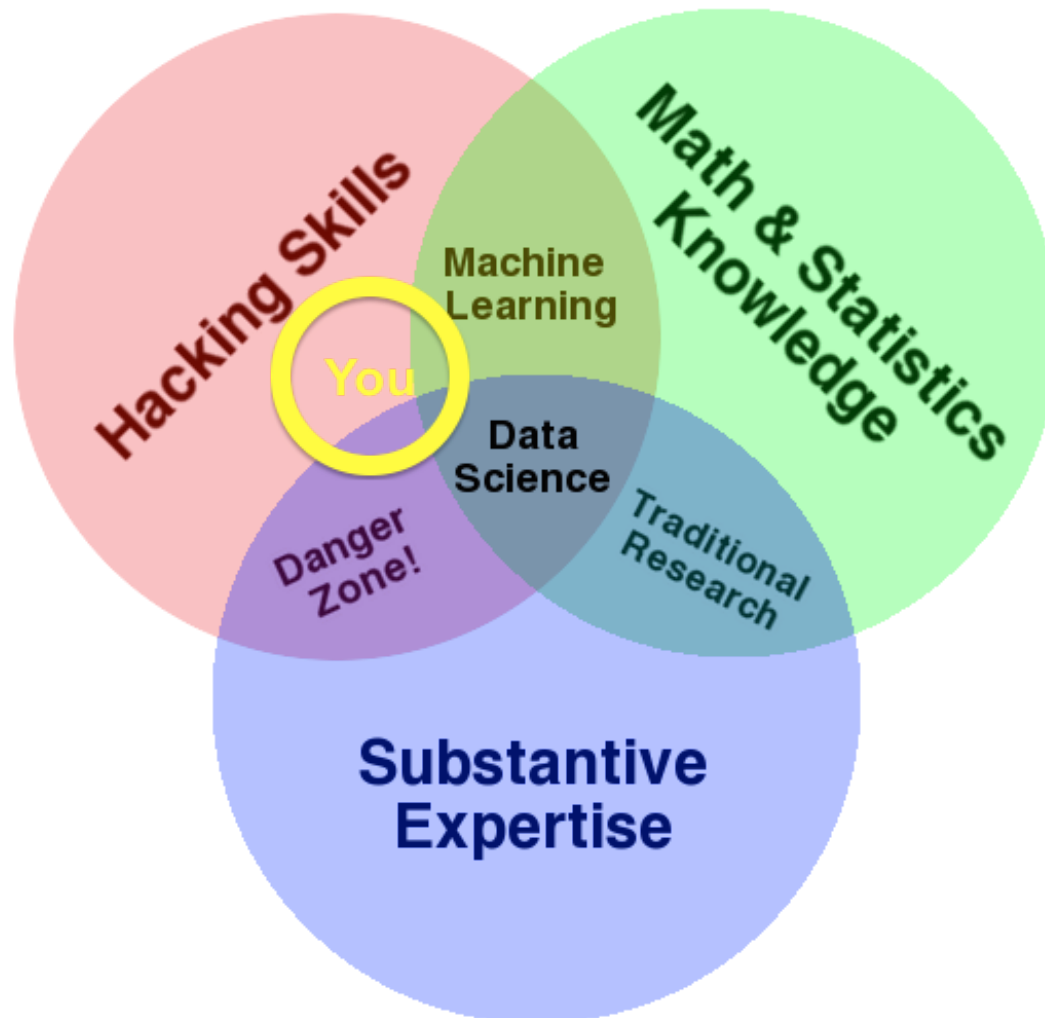
A few more articles on the risks:

- [Surviving Data Science "at the Speed of Hype"](#) by John Foreman, Data Scientist at MailChimp

- The High Cost of Maintaining Machine Learning Systems
- 11 Clever Methods of Overfitting and How to Avoid Them
- The Perilous World of Machine Learning for Fun and Profit: Pipeline Jungles and Hidden Feedback Loops

Welcome to the Danger Zone

So you are dabbling with Machine Learning. You've got Hacking Skills. Maybe you've got some "knowledge" in Domingos' sense (some "Substantive Expertise" or "Domain Knowledge"). This diagram is modified slightly from Drew Conway's "Data Science Venn Diagram." It isn't a *perfect* fit for us, but it may get the point across:



Please don't sell yourself as a Machine Learning expert while you're still in the Danger Zone. Don't build bad products or publish junk science. (Also please [don't be evil](#).) This guide can't tell you how you'll know you've "made it" into Machine Learning competence ... let alone expertise. It's hard to evaluate proficiency without schools or other institutions. This is a common problem for self-taught people.

Towards Expertise

You need practice. On Hacker News, user olympus commented to say you could use competitions to practice and evaluate yourself. Kaggle and ChaLearn are hubs for Machine Learning competitions. You can find some examples of code for popular Kaggle competitions [here](#). For smaller exercises, [try HackerRank](#).

You also need understanding. You should review what Kaggle competition winners say about their solutions, for example, the ["No Free Hunch" blog](#). These might be over your head at first but once you're starting to understand and appreciate these, you know you're getting somewhere.

Competitions and challenges are one way to practice. You shouldn't limit yourself, though.

Here's a complementary way to practice: **do practice studies**.

1. **Ask a question. Start your own study.** The ["most important thing in data science is the question"](#) (Dr. Jeff T. Leek). So start with a question. Then, find [real data](#). Analyze it. Then ...
2. **Communicate results.** When you have a novel finding, reach out for peer review.
3. **Fix issues.** Learn. Share what you learn.

And repeat. Re-phrasing this, it fits with the [scientific method](#): formulate a question (or problem statement), create a hypothesis, gather data, analyze the data, and communicate results. (You should [watch this video about the scientific method in data science](#), and/or [read this article](#).)

How can you come up with interesting questions? Here's one way. Every Sunday, [browse datasets](#) and write down some questions. Also, sign up for [Data is Plural](#), a newsletter of interesting datasets; look at these, datasets, and write down questions. Stay curious. When a question inspires you, start a study.

This advice, to do practice studies and learn from peer review, is based on [a conversation with Dr. Randal S. Olson](#). Here's more advice from Olson, [quoted with permission](#):

I think the best advice is to tell people to always present their methods clearly and to avoid over-interpreting their results. Part of being an expert is knowing that there's rarely a clear answer, especially when you're working with real data.

As you repeat this process, your practice studies will become more scientific, interesting, and focused. The most important part of this process is peer review.

Ask for Peer Review

Here are some communities where you can reach out for peer review:

- Cross-Validated: stats.stackexchange.com
- [/r/DataIsBeautiful](https://www.reddit.com/r/DataIsBeautiful/)
- [/r/DataScience](https://www.reddit.com/r/DataScience/)
- [/r/MachineLearning](https://www.reddit.com/r/MachineLearning/)
- Hacker News: news.ycombinator.com. You'll probably want to submit as "Show HN"

Post to any of those, and ask for feedback. You'll get feedback. You'll learn a ton. As experts review your work you will learn a lot about the field. You'll also be practicing a crucial skill: accepting critical feedback.

When I read the feedback on my Pull Requests, first I repeat to myself, "I will not get defensive, I will not get defensive, I will not get defensive." You may want to do that before you read reviews of your Machine Learning work too.

An Anecdote About User Experience

If you create software for users, and you want to use machine learning to benefit your users, *you must understand your users*. I won't get into a whole user experience rant here, but in short, you must think about user experience.

I have a friend who worked at <Redacted> Music Streaming Service. This company used machine learning in their recommendation and radio services. He complained about the way the company scored the radio feature's performance. There was disagreement about what should be scored. They used a metric, "no song skips." But why? Sure that indicates the recommendation wasn't *awful*, what if you want to measure engagement? Other metrics could measure positive engagement: "favorites," shares, listening time, or whether the listener returns to the radio station later. Measuring "no skips" might work for

the passive listener, but the engaged listener is different. Perhaps the engaged listener will skip 5 songs, but find 20 songs they love and come back to the service later.

My takeaway: user experience matters just as much as ever. You must understand *which kind* of user you're trying to benefit with your machine learning techniques. Write user stories. (Some readers may notice this anecdote is very unscientific. Well, user stories are sometimes unscientific. Yet still so important.)

Try to measure success *for your users*. You must be able to measure before you can optimize.

It helps to have knowledge of the domain (substantive expertise). You may also benefit from UX expertise. Work with UX experts if you can.

Machine Learning in Internet Security

There was a great BlackHat webcast on this topic, [Secure Because Math: Understanding Machine Learning-Based Security Products](#). Slides are [here](#), [video recording is here](#). Equally relevant to InfoSec and AppSec.

Big Data?

Scaling data analysis is a familiar problem now, and there's no shortage of ways to address it. [Beware needless hype and companies selling you flashy, proprietary solutions](#). You can do it all with open-source tools. Even if "buy" instead of "build," you may want to buy from vendors who use known good stacks. No news here.

Here are some open-source tools to reach for:

- [Apache Spark](#).
 - I mean, hell, [Spark has DataFrames and easy co-operability with pandas!](#)
 - [Berkeley has a course on Scalable Machine Learning, focusing on Apache Spark.](#)
- [NetflixOSS](#) (see "Big Data Tools")

- [Netflix: Surus and ScorePMML](#)
- ["big data" on the NetflixOSS blog](#)

Also: [10 things statistics taught us about big data analysis](#)

Finding Open-Source Libraries

- Bookmark [awesome-machine-learning](#), a curated list of [awesome](#) Machine Learning libraries and software.
- [TensorFlow](#) seems like a [really big deal](#). People like you will do exciting things with TensorFlow. It's a framework. Frameworks can help you manage complexity. Just remember this rule of thumb: **"More data beats a cleverer algorithm"** (Domingos), no matter how cool your tools are. Also note, TensorFlow is not the only machine learning framework of its kind: [Check this great, detailed comparison of TensorFlow, Torch, and Theano](#). See also [Newmu/Theano-Tutorials](#) and [nlintz/TensorFlow-Tutorials](#).
- Bookmark [Pythonidae](#), a curated list of [awesome](#) libraries and software in the Python language - with a section on Machine Learning.
- Bookmark [Julia.jl](#), a curated list of [awesome](#) libraries and software in the Julia language - with a section on Machine Learning.
- For Machine-Learning libraries that might not be on PyPI, GitHub, etc., there's [MLOSS \(Machine Learning Open Source Software\)](#). Seems to feature many academic libraries.

Alternative ways to "Dive into Machine Learning"

Here are some other guides to Machine Learning. They can be alternatives or complements to this guide.

- [Machine Learning for Developers](#) is another good introduction. It introduces machine learning for a developer audience using Smile, a machine learning library that can be used both in Java and Scala.
- [Materials for Learning Machine Learning](#) by Jack Simpson

- [How to Machine Learn](#) by Gideon Wulfsohn
- [Machine Learning for Programmers \[with caveats\]](#): Pragmatic guide for developers, definitely worth a read. I've linked you to my own intro to it, explaining a few caveats. Taking it with a grain of salt, and these caveats in mind, I think some programmers will find it worth their time.
- [Example Machine Learning notebook, exercise, and guide](#) by Dr. Randal S. Olson. Mentioned in Notebooks section as well, but it has a similar goal to this guide (introduce you, and show you where to go next). Rich "Further Reading" section.
- ["How would your curriculum for a machine learning beginner look like?"](#) by Sebastian Raschka. A selection of the core online courses and books for getting started with machine learning and gaining expert knowledge.
- The [pattern_classification](#) [GitHub repository](#) maintained by Sebastian Raschka, which contains IPython notebooks about various machine learning algorithms and various data science related resources.
- [Your guide here]